SSM-Aware Fine-Tuning for Hybrid Mamba-Transformer Models: A Comparative Study on Granite 4.0-H-Micro

Cody Ford
HarmoniqOS
February 2026

Abstract

We present a systematic study of LoRA fine-tuning strategies for IBM Granite 4.0-H-Micro, a 3.2B-parameter hybrid architecture comprising 36 Mamba-2 state space layers and 4 Transformer attention layers. We evaluate four fine-tuning approaches against an unmodified baseline across three domain-specific tasks: document classification (24K examples), schema mapping (15K examples), and structured rule generation (9K examples). Our investigation proceeds in two stages. First, we find that co-training LoRA adapters with unfrozen SSM core parameters (A_log, D, dt_bias) yields consistent improvements across all tasks (V3). However, PEFT's adapter-only serialization — combined with a save-ordering issue in our training script — silently discarded the trained SSM values from the saved PEFT artifact. Second, after fixing the persistence pipeline (V4), we estimate the SSM parameters' direct contribution: an additional 3.6 percentage point gain on classification (55.8% vs 52.2%), confirming that the co-training effect and persistent SSM adaptation are complementary mechanisms. Classification benefits most from persistent SSM changes — a cumulative 37% relative improvement over LoRA-only (V2) — while schema mapping and rule generation gains are driven primarily by the co-training effect alone. We are not aware of prior public results that combine LoRA targeting of Mamba projections with training and persisting SSM core parameters on a publicly available hybrid Mamba-Transformer model.

1. Introduction

Hybrid architectures combining Mamba-2 selective state space models (SSMs) with Transformer attention layers represent an emerging class of language models that achieve subquadratic sequence processing while retaining the in-context learning capabilities of attention. IBM's Granite 4.0-H-Micro exemplifies this design: 36 of its 40 layers use Mamba-2's structured state space duality (SSD) formulation, with attention layers placed at regular intervals (positions 5, 15, 25, 35).

Parameter-efficient fine-tuning (PEFT) methods like LoRA have become standard for adapting large language models to downstream tasks. However, the LoRA literature overwhelmingly targets pure Transformer architectures, with `target_modules` typically set to attention projections (q/k/v/o_proj) and MLP layers (gate/up/down_proj). When applied to hybrid models, this default configuration misses the Mamba-specific projection layers entirely, potentially leaving 90% of the model's representational capacity untouched.

This work originated as an engineering investigation while fine-tuning Granite for a domain-specific automation product. It addresses four questions:

1. Target selection matters: Does targeting the correct projection layers (Mamba in_proj/out_proj, shared MLP input_linear/output_linear) produce meaningfully better results than default Transformer targets?

2. SSM core parameters: Can unfreezing the Mamba-2 core state parameters (A_log, D, dt_bias) during LoRA training further improve task performance?

3. Co-training vs. persistence: If SSM parameter changes are not preserved in the final model (due to PEFT serialization limitations), does the co-training still provide measurable benefit?

4. Isolating the SSM contribution: Once the persistence issue is fixed, how much additional improvement comes from the SSM parameter values themselves versus the indirect co-training effect on LoRA weights?

2. Model Architecture

2.1 Granite 4.0-H-Micro

Parameter	Value
Architecture class	GraniteMoeHybrid (Mamba-2 / Transformer)
Total parameters	3,220,219,648 (~3.2B)
Hidden dimension	2048
Intermediate dimension (MLP)	8192
Vocabulary size	100,352
Total layers	40
Mamba-2 layers	36 (positions 0–4, 6–14, 16–24, 26–34, 36–39)
Attention layers	4 (positions 5, 15, 25, 35)
Attention heads	32 (8 KV heads, GQA 4:1)
Precision	bfloat16

The attention layers are evenly spaced at every 10th position, creating a pattern where Mamba-2 layers handle the majority of sequence processing while attention layers provide periodic global information mixing.

2.2 Mamba-2 Layer Configuration

Parameter	Value
State dimension (d_state)	128
Head dimension (d_head)	64
Number of heads (n_heads)	64
Groups	1
Expand factor	2
Convolution kernel size	4
Chunk size	256

Each Mamba-2 layer contains three core SSM parameters:

• **A_log** [shape: 64]: Log of the state transition diagonal — controls how information decays across sequence positions

• **D** [shape: 64]: Skip connection weight — determines how much of the input bypasses the state space recurrence

• **dt_bias** [shape: 64]: Timestep discretization bias — influences the effective step size for state updates

These are 1D tensors (one scalar per head) rather than full matrices, a consequence of Mamba-2's structured state space duality (SSD) simplification where the state transition matrix is constrained to a scalar-times-identity form per head.

2.3 Trainable Module Landscape

The model presents the following linear projection layers available for LoRA adaptation:

Module	Location	Count	Purpose
q_proj	Attention layers	4	Query projection
k_proj	Attention layers	4	Key projection
v_proj	Attention layers	4	Value projection
o_proj	Attention layers	4	Output projection
in_proj	Mamba-2 layers	36	SSM input projection
out_proj	Mamba-2 layers	36	SSM output projection
input_linear	All layers (shared MLP)	40	MLP up-projection
output_linear	All layers (shared MLP)	40	MLP down-projection

Critically, the standard Transformer MLP module names (`gate_proj`, `up_proj`, `down_proj`) **do not exist** in this architecture. The Granite hybrid uses a shared MLP with `input_linear` and `output_linear` instead.

3. Training Data

3.1 Synthetic Data Generation Pipeline

All training data was generated synthetically using an LLM-based pipeline operating on a catalog of 611 domain-specific document schemas spanning over 10,000 structured query rules and 7,000+ unique column definitions. The schemas represent tabular data export formats from a variety of enterprise software tools.

The generation pipeline employed:

- Structured JSON output constraints
- Concurrent API workers with retry logic and truncated JSON repair
- Explicit diversity quotas per variation type within each prompt
- Idempotent per-schema processing enabling restartable pipeline runs

3.2 Task Descriptions and Dataset Statistics

Classification (24,369 examples)

Task: Given tab-separated column headers and 2–3 rows of sample data from a tabular document, predict the correct schema label from a vocabulary of 611 possible labels.

Data generation: For each schema, 40 synthetic document examples were generated across 10 variation types:

Variation Type	Examples	Description
full	2,436	All columns, standard naming
partial	2,443	Subset of columns, missing optional fields

Variation Type	Examples	Description
renamed	3,627	Name variations across software versions
minimal	2,440	Bare minimum identifying columns
shuffled	2,437	Same columns in randomized order
mixed_case	2,434	Casing variants (lowercase, CAPS, camelCase)
extra_columns	2,439	Standard columns plus additional extras
api_export	2,434	snake_case / dot.notation from REST APIs
legacy	1,837	Column names from older software versions
confusable	1,842	Headers resembling a different schema

Example format:

```
Instruction: "Classify this document and return the schema label."
Input:       "col_a\tcol_b\tcol_c\t...\nval1\tval2\tval3\t..."
Output:      "schema_label_123"
```

Schema Mapping (15,222 examples)

Task: Given a list of document column headers and a list of expected schema columns, produce a JSON mapping from document headers to expected columns, identifying any unmapped columns.

Data generation: For each schema, 25 mapping scenarios across 8 types (standard, api_export, truncated, renamed, mixed_case, extra_columns, ambiguous, different_naming).

Example format:

```
Instruction: "Map document columns to expected schema columns."
Input:       {"file_headers": ["col_a", ...], "expected_columns": ["column_a", ...]}
Output:      {"mapping": {"col_a": "column_a", ...}, "unmapped_expected": []}
```

Structured Rule Generation (9,165 examples)

Task: Given a schema label, available columns, and a pattern type, generate a structured query rule as JSON including a SQL WHERE clause, severity, category label, description, and domain-specific metadata.

Data generation: For each schema, 15 rules across 9 pattern types (pattern_match, missing_fields, date_staleness, status_anomaly, configuration, access_control, severity_threshold, authentication, encryption). Up to 3 existing query rules per schema were provided as few-shot examples.

Example format:

```
Instruction: "Generate a detection rule for this data schema."
Input:       {"schema_label": "schema_label_123", "columns": [...],
              "pattern_type": "status_anomaly"}
Output:      {"rule_type": "inactive_accounts", "severity": "HIGH",
              "description": "...", "where_clause": "...",
              "metadata_ids": ["ID-1", "ID-2"]}
```

3.3 Train/Test Split

All datasets use a deterministic 90/10 split (seed=42). The evaluation set is held out consistently across all experiments.

Task	Training Examples	Evaluation Examples
Classification	21,932	2,437

Task	Training Examples	Evaluation Examples
Schema Mapping	13,699	1,523
Rule Generation	8,248	917
Total	**43,879**	**4,877**

4. Experimental Setup

4.1 Training Configurations

We evaluate five conditions:

Version	Description	Target Modules	Trainable Params	% of Model
Base	No fine-tuning	—	0	0%
V1	Wrong LoRA targets	q/k/v/o_proj, gate/up/down_proj	851,968	0.027%
V2	Correct LoRA targets	q/k/v/o_proj, in/out_proj, input/output_linear	28,823,552	0.895%
V3	Correct LoRA + SSM co-training (SSM lost)	Same as V2 + unfrozen A_log, D, dt_bias	28,830,464	0.895%
V4	Correct LoRA + SSM co-training (SSM persisted)	Same as V3, fixed save pipeline	28,830,464	0.895%

V1 uses the default Transformer LoRA targets that are standard in the literature. Because `gate_proj`, `up_proj`, and `down_proj` do not exist in Granite's architecture, only the 4 attention layers' q/k/v/o projections receive LoRA adapters — a total of 16 adapted weight matrices covering 0.027% of model parameters.

V2 targets the correct architecture-specific modules: Mamba-2 projections (`in_proj`, `out_proj`), shared MLP (`input_linear`, `output_linear`), and attention projections (q/k/v/o_proj). This covers all 40 layers with 168 adapted weight matrices, a **33.8x increase** in trainable parameters over V1.

V3 uses identical LoRA configuration to V2, adding only 6,912 directly-unfrozen SSM core parameters (64 heads x 3 params x 36 Mamba layers). These are standard PyTorch `requires_grad = True` parameters that participate in backpropagation alongside the LoRA gradients. However, V3's trained SSM parameters were silently discarded by PEFT's serialization (see Section 6.3), meaning V3's deployed model contains only the LoRA adaptations — the SSM values reverted to base model values.

V4 uses the same training configuration as V3 with a corrected save pipeline that captures SSM core parameters before PEFT's `save_pretrained()` call resets `requires_grad` flags. The trained SSM values are saved to a separate `ssm_core_params.pt` checkpoint and injected into the merged model during GGUF conversion. To our knowledge, this configuration has not been previously evaluated — V4 is the only version where both LoRA adaptations and SSM modifications are present in the deployed model.

4.2 Common Hyperparameters

All three fine-tuned versions use identical training hyperparameters:

Parameter	Value
LoRA rank (r)	16
LoRA alpha	32

Parameter	Value
LoRA scaling factor	alpha/r = 2.0
LoRA dropout	0.05
Epochs	3
Learning rate	2e-4
LR scheduler	Cosine with warmup
Warmup ratio	0.06
Weight decay	0.01
Max gradient norm	1.0
Max sequence length	1280 tokens
Effective batch size	32
Precision	bfloat16 (full, no quantization)
Gradient checkpointing	Enabled
Example packing	Enabled (SFTTrainer)
Evaluation	Every 50 steps
Model selection	Best eval_loss

Training was conducted on an NVIDIA RTX PRO 6000 GPU via Vast.ai cloud compute. Full bfloat16 precision was used rather than QLoRA (4-bit quantization) because bitsandbytes quantized weights are incompatible with Mamba-2's fast Triton kernels due to shape mismatches in `F.linear` calls. At ~2GB in bf16, the model fits comfortably in RTX PRO 6000 memory without quantization.

4.3 Mamba-2 Compatibility

Loading Granite 4.0-H-Micro requires a mandatory monkey-patch to expose Triton kernel operations on the `mamba_ssm` namespace before model import:

```
from mamba_ssm.ops.triton.selective_state_update import selective_state_update
from mamba_ssm.ops.triton.ssd_combined import (
    mamba_split_conv1d_scan_combined, mamba_chunk_scan_combined
)
import mamba_ssm
mamba_ssm.selective_state_update = selective_state_update
mamba_ssm.mamba_chunk_scan_combined = mamba_chunk_scan_combined
mamba_ssm.mamba_split_conv1d_scan_combined = mamba_split_conv1d_scan_combined
```

This patches three Triton kernel operations that HuggingFace Transformers expects to find when loading `granitehybrid` architecture models.

4.4 V3 SSM Co-Training Implementation

After PEFT wraps the model with LoRA adapters, SSM core parameters are unfrozen:

```
SSM_CORE_PARAMS = ["mamba.A_log", "mamba.D", "mamba.dt_bias"]

def apply_ssm_core_training(model):
    for name, param in model.named_parameters():
        if any(x in name for x in SSM_CORE_PARAMS):
            param.requires_grad = True

model = get_peft_model(model, lora_config)
```

```
apply_ssm_core_training(model)
model.enable_input_require_grads()
```

The `enable_input_require_grads()` call is required when mixing PEFT adapters with directly unfrozen parameters to ensure proper gradient propagation.

4.5 Persisting SSM Core Parameters with PEFT

V3's failure to preserve SSM parameters was caused by two interacting issues:

1. PEFT serialization limitation: `model.save_pretrained()` only serializes LoRA adapter weights (A/B matrices). Non-LoRA parameter modifications are silently discarded — no warning is emitted.

2. Training script save-ordering issue: The SSM parameter save code ran *after* `trainer.save_model()`, which internally calls PEFT's `save_pretrained()`. This switches the PEFT wrapper into inference mode, and non-adapter parameters no longer appear as trainable. The save code filtered on `requires_grad`, finding nothing to save.

V4 fixes both issues:

```
# V3 (broken): save_model() first, then try to find SSM params
trainer.save_model(str(final_dir))          # sets inference_mode
ssm_state = {n: p for n, p in model.named_parameters()
             if any(x in n for x in SSM_CORE_PARAMS)
             and p.requires_grad}            # empty!

# V4 (fixed): save SSM params first, match on name only
ssm_state = {}
for name, param in model.named_parameters():
    if any(x in name for x in SSM_CORE_PARAMS):
        ssm_state[name] = param.detach().cpu().clone()
torch.save(ssm_state, final_dir / 'ssm_core_params.pt')
trainer.save_model(str(final_dir))           # LoRA saved separately
```

Verification on the training instance confirmed all 108 SSM parameters (36 layers x 3 params) differed from base model values (mean absolute difference ~0.0004 for A_log).

4.6 Model Deployment Pipeline

For evaluation, models are deployed as GGUF files via llama-cpp-python:

- **V1**: Runtime LoRA application — base model GGUF (Q4_K_M, 1.9GB) with separate LoRA adapter GGUF (F16, ~1.7MB)
- **V2/V3**: Pre-merged GGUF — LoRA weights baked into base model tensors via `W_merged = W_base + (alpha/r) * (B @ A)`, then quantized to Q8_0 (~3.2GB each)
- **V4**: Two-step merge — LoRA weights merged as in V2/V3, then SSM core parameters injected from `ssm_core_params.pt` by replacing base A_log, D, and dt_bias tensors with trained values

The LoRA merge uses pure tensor math (no model loading required):

```
scale = alpha / r  # 32 / 16 = 2.0
for each LoRA target:
    A = lora_tensors[key + '.lora_A']  # [r, in_features]
    B = lora_tensors[key + '.lora_B']  # [out_features, r]
    W_merged = W_base + scale * (B @ A)
```

The SSM injection is a direct tensor replacement:

```
for peft_name, trained_tensor in ssm_core_params.items():
    base_key = peft_name.replace("base_model.model.", "")
    merged_tensors[base_key] = trained_tensor.to(base_dtype)
```

All computation is performed in float32 with results cast back to the base tensor's original dtype (bfloat16) before GGUF conversion.

5. Evaluation Methodology

5.1 Inference Configuration

All models are evaluated using llama-cpp-python with full GPU offloading (NVIDIA GeForce RTX 4060 Ti):

- Context window: 2048 tokens
- Temperature: 0.0 (greedy decoding)
- Stop tokens: `<|end_of_text|>`, `<|start_of_role|>`
- KV cache reset between examples via `llm.reset()`
- Max generation: 64 tokens (classification), 512 tokens (schema mapping, rule generation)

5.2 Prompt Format

The Granite native chat template is used:

```
&lt;|start_of_role|&gt;system&lt;|end_of_role|&gt;
You are a domain-specific analysis assistant.&lt;|end_of_text|&gt;
&lt;|start_of_role|&gt;user&lt;|end_of_role|&gt;
{instruction}\n\n{input}&lt;|end_of_text|&gt;
&lt;|start_of_role|&gt;assistant&lt;|end_of_role|&gt;
```

5.3 Scoring Functions

Classification: Exact match (model output, first line, lowercased, stripped of special tokens == expected label) and contains match (expected label appears anywhere in output).

Schema Mapping: JSON validity (whether output parses as valid JSON), mapping accuracy (per-field exact match of predicted mapping vs. expected mapping), and unmapped accuracy (set overlap of unmapped columns).

Structured Rule Generation: JSON validity, structural completeness (presence of all required fields), severity correctness (case-insensitive match), and category label correctness.

5.4 Evaluation Scale

Final results are computed over **1,000 test examples per task** drawn from the held-out 10% evaluation split. The same random seed (43) ensures identical test sets across all model versions.

6. Results

6.1 Primary Results (n=1,000 per task)

Classification

Model	Exact Match	Contains Match	Throughput
Base (no training)	0.0%	0.1%	1.02 ex/s
V1 (wrong targets)	0.1%	0.1%	1.95 ex/s
V2 (correct LoRA)	40.8%	45.9%	4.09 ex/s
V3 (LoRA + SSM, lost)	52.2%	53.8%	4.24 ex/s

Model	Exact Match	Contains Match	Throughput
V4 (LoRA + SSM, persisted)	**55.8%**	**58.4%**	**4.17 ex/s**

Schema Mapping

Model	JSON Valid	Mapping Accuracy	Unmapped Accuracy
Base (no training)	1.9%	0.48%	1.2%
V1 (wrong targets)	0.4%	0.0%	0.3%
V2 (correct LoRA)	99.9%	96.03%	96.64%
V3 (LoRA + SSM, lost)	99.9%	97.65%	97.55%
V4 (LoRA + SSM, persisted)	**99.5%**	**96.73%**	**97.28%**

Structured Rule Generation

Model	JSON Valid	All Fields	Severity	Category Label
Base (no training)	29.99%	0.0%	3.27%	0.0%
V1 (wrong targets)	67.94%	0.0%	0.11%	0.0%
V2 (correct LoRA)	98.58%	98.15%	39.91%	2.73%
V3 (LoRA + SSM, lost)	98.91%	98.80%	41.33%	3.16%
V4 (LoRA + SSM, persisted)	**98.8%**	**97.49%**	**40.57%**	**3.05%**

Training Loss (Classification Task)

Model	Eval Loss
V1 (wrong targets)	1.0853
V2 (correct LoRA)	0.6339
V3 (LoRA + SSM)	**0.5876**

6.2 Analysis: Target Selection (V1 → V2)

V1 demonstrates that default LoRA targets are catastrophically ineffective on hybrid architectures. With only 851,968 trainable parameters (0.027% of the model) confined to 4 attention layers, V1 performs no better than the untrained base model on classification and mapping — and actually performs *worse* on mapping (0.0% vs. 0.48%). The model's capacity to learn task-specific behavior is almost entirely determined by the 36 Mamba-2 layers that V1 leaves completely untouched.

V2 confirms that architecture-aware target selection is essential. The 33.8x increase in trainable parameters (from targeting the correct modules) produces dramatic improvements: 0% to 40.8% classification accuracy, 0% to 96% mapping accuracy, and structured JSON output improving from 30% to 98.6% validity. This is the most consequential design decision in the entire pipeline.

6.3 The Co-Training Effect (V2 → V3)

V3 was designed to produce a model with both LoRA adaptations and modified SSM core parameters baked into the final weights. **The SSM parameters were silently discarded due to PEFT's adapter-only**

serialization behavior (detailed in Section 4.5): PEFT only saves LoRA adapter weights when calling `model.save_pretrained()`, and a training script ordering issue prevented the separate SSM checkpoint from capturing the trained values. We verified empirically that V3's merged safetensors contain SSM parameter values identical to the base model (zero diff across all 6,912 elements).

Despite this, V3 consistently outperforms V2 across all metrics. We attribute this to a co-training effect: during training, the unfrozen SSM parameters participate in backpropagation, creating gradient pathways through the state space recurrence that influence how the LoRA adapter weights are updated. The SSM parameters act as auxiliary optimization variables that reshape the loss landscape, enabling the LoRA weights to converge to qualitatively different (and better) solutions than they would if the SSM parameters were frozen. This is analogous to auxiliary loss functions in multi-task learning — the auxiliary signal improves the primary task even though the auxiliary outputs are discarded at inference time.

V3's largest gains over V2 appear on classification (+11.4 percentage points, a 28% relative improvement) and mapping accuracy (+1.62 percentage points, a 41% error rate reduction). The co-training effect is consistent but task-dependent: classification benefits most, while structured output tasks (schema mapping, rule generation) show smaller improvements.

6.4 Isolating the SSM Contribution (V3 → V4)

V4 fixes the persistence pipeline (Section 4.5) and represents the only configuration in our study where both LoRA adaptations and trained SSM parameters are present in the deployed model. Comparing V4 to V3 provides an estimate of the direct contribution of SSM parameter values at inference time, since both trained with identical configurations; the key difference is that only V4 preserves the SSM changes.

Classification: SSM persistence provides additional gains. V4 achieves 55.8% exact match versus V3's 52.2%, a 3.6 percentage point improvement (+6.9% relative). The cumulative improvement from V2 (LoRA only) to V4 (LoRA + persistent SSM) is 15.0 percentage points (+36.8% relative). This decomposes into:

- Co-training effect (V2→V3): +11.4 points (76% of total gain)
- SSM persistence (V3→V4): +3.6 points (24% of total gain)

The `contains_match` metric shows an even larger V4 gain: 58.4% vs 53.8% (+4.6 points), suggesting the persistent SSM parameters improve the model's ability to produce the correct label even when additional text is generated around it.

Schema mapping and rule generation: Co-training dominates, persistence is neutral. V4 mapping accuracy (96.73%) is marginally below V3 (97.65%), and rule generation metrics are essentially flat (98.8% vs 98.91% JSON valid). Both remain well above V2. The V3→V4 differences are within normal evaluation variance and do not suggest that persistent SSM parameters hurt these tasks — rather, the structured output tasks derive their improvement primarily from the co-training effect on LoRA weights, with the actual SSM parameter values contributing minimally at inference time.

Interpretation: Classification requires discriminating between 611 possible labels based on sequence-level patterns in column headers and sample data — exactly the kind of sequential pattern recognition that Mamba-2's state dynamics (A_log decay rates, D skip connections) directly control. Schema mapping and rule generation are structured output tasks where the model has already learned the correct JSON format; the remaining accuracy is determined by field-level precision, which depends more on the projection layer weights (adapted by LoRA) than on state transition dynamics.

Parameter budget perspective: The SSM co-training adds only 6,912 parameters (0.024% of V3/V4's trainable budget). In V3, these parameters have zero inference cost since they are discarded. In V4, persisting them adds zero parameter overhead — the parameters already exist in the base model, only their values change.

6.5 Throughput

V2, V3, and V4 merged models achieve 4.0+ examples/second on classification tasks on a consumer-grade GPU (RTX 4060 Ti), approximately 2x faster than V1's runtime LoRA application. V4's SSM parameter injection adds zero inference overhead since the parameters already exist in every Mamba-2 layer — only their values differ. For schema mapping and rule generation tasks requiring longer outputs (512 max tokens), all versions converge to ~0.4 ex/s, limited by token generation speed rather than model loading.

7. Discussion

7.1 Implications for Hybrid Model Fine-Tuning

The stark difference between V1 and V2 results highlights a practical gap in the LoRA ecosystem. Most LoRA tutorials, configuration defaults, and automated tools assume pure Transformer architectures. When applied to hybrid models without modification, they silently fail to adapt the model's dominant computation pathway. We recommend that LoRA practitioners inspect their target model's `named_modules()` output before training, rather than relying on default module name lists.

7.2 PEFT's Silent Module-Skipping Behavior

A critical tooling failure directly caused V1's near-zero results: **PEFT does not warn when `target_modules` matches zero or near-zero modules in the model.** When V1 specified `gate_proj`, `up_proj`, and `down_proj` — standard Transformer MLP targets — PEFT silently skipped all 36 Mamba-2 layers (which use `input_linear` and `output_linear` instead), attaching LoRA adapters only to the 4 attention layers' q/k/v/o projections. The training proceeded without error, loss decreased nominally, and the adapter saved successfully — giving no indication that 90% of the model was untouched.

This silent skipping behavior is particularly dangerous because:

1. **No warning is emitted** when specified module names don't match any modules
2. **Training still converges** (loss decreases), masking the misconfiguration
3. **The resulting adapter files are valid**, deferring failure detection to downstream evaluation
4. **The failure mode is subtle**: V1 doesn't produce obviously broken output — it just doesn't improve over the base model

We consider this a tooling deficiency that the PEFT library should address, either by warning when `target_modules` matches fewer modules than expected, or by requiring explicit acknowledgment when the match rate is unusually low (e.g., <10% of linear layers).

7.3 SSM Parameters as Training Regularizers

The co-training effect observed in V3 — where SSM parameters improved LoRA convergence despite being discarded — suggests that SSM core parameters (A_log, D, dt_bias) function as implicit regularizers during LoRA fine-tuning. By allowing gradients to flow through the state space recurrence, the optimization process gains access to richer gradient information about how sequence-level representations are formed. This may prevent the LoRA adapters from overfitting to superficial input-output patterns, instead learning adaptations that work with the model's native sequential processing.

V4's results strengthen this interpretation. The co-training effect (V2→V3) accounts for 76% of the total classification improvement, while SSM persistence (V3→V4) accounts for only 24%. If the SSM parameters were the primary driver, we would expect V4 to show a much larger jump over V3. Instead, the dominant mechanism is the indirect effect on LoRA optimization, with the persistent SSM values providing a complementary but smaller direct benefit.

Training logs reveal a consistent hierarchy across all three tasks (Appendix A.3): **D** (direct feedthrough) changes the most (39–94% of elements), **A_log** (state transition) changes moderately (19–48%), and **dt_bias** (timestep discretization) is effectively frozen at 0.0% across all tasks and layers, with a single exception (rule generation, layer 1, 3.1%). This pattern holds regardless of task type, suggesting it reflects a structural property of the Mamba-2 SSD parameterization rather than a task-specific effect.

The **D** parameter's dominance is notable: it controls the skip connection that determines how much raw input bypasses the recurrent state computation. The model preferentially tunes this direct feedthrough path during co-training, suggesting that adjusting the balance between recurrent and feedforward processing is the primary mechanism by which SSM unfreezing improves LoRA optimization.

The **dt_bias** freeze is likely caused by gradient suppression through the exponential activation applied during the Mamba-2 forward pass (`dt = F.softplus(dt_bias + dt)`). Per-parameter learning rate schedules or alternative parameterizations (e.g., removing the softplus during fine-tuning) could unlock additional capacity.

V1 training logs provide a control: with SSM parameters frozen, all three parameters show exactly 0.0% change across all tasks, confirming the V3/V4 deltas are genuine training effects rather than numerical noise.

7.4 Task-Dependent SSM Sensitivity

V4 reveals that SSM persistence benefits are task-dependent. Classification — which requires discriminating between 611 labels from sequence-level patterns — benefits from persistent SSM changes (+3.6 points). Schema mapping and rule generation — structured output tasks where the model has already learned the correct format — show no meaningful change from SSM persistence.

A paradox emerges from the training logs (Appendix A.3): rule generation actually drives the *strongest* SSM parameter changes during training — D at layer 0 changes 68.8% for code generation vs 46.9% for classification — yet shows *no benefit* from SSM persistence at inference. This suggests that for structured output tasks, the SSM parameters function purely as optimization scaffolding: they actively shape gradient flow during training (explaining the co-training effect) but their final values carry no useful information beyond what the LoRA weights have already captured. Classification, by contrast, encodes task-relevant information *in* the SSM values themselves — specifically, how aggressively to decay state across positions and how much to weight direct input vs recurrent context.

This aligns with the functional roles of the SSM core parameters: A_log controls information decay across sequence positions (how quickly the model "forgets" earlier tokens), and D controls the skip connection weight (the balance between recurrent state and direct input). For classification, the model must integrate information across the entire input sequence (column headers + sample rows) to select one label from 611 options — a task where modified decay and feedthrough dynamics provide a direct advantage. For structured output tasks, the model primarily needs to map local patterns (individual column names, field values) to output tokens, where the base model's SSM dynamics are already sufficient.

7.5 Limitations

- **Single model**: Results are reported on one model (Granite 4.0-H-Micro). Generalization to other hybrid architectures (e.g., Jamba, Zamba) requires further study.
- **Synthetic data**: All training and evaluation data is synthetically generated. Real-world data may exhibit different distributions.
- **V3/V4 not independently seeded**: V3 and V4 used the same training configuration but were trained in separate runs. Minor differences between V3 and V4 on mapping/code_generation may reflect run-to-run variance rather than SSM persistence effects. Multiple seeds would strengthen the V3→V4 comparison.

• **Quantization mismatch**: Base model evaluation uses Q4_K_M quantization while V2/V3/V4 merged models use Q8_0. Both are quantized from the same bfloat16 source weights, but the different quantization levels may introduce minor systematic differences. However, V1 (which uses the same Q4_K_M base with runtime LoRA) performs at or below the unmodified base on several metrics, suggesting that quantization level is not the primary driver of the V2+ improvements.

7.6 Future Work

1. Per-parameter learning rates: Applying different learning rates to LoRA adapters vs. SSM core parameters to optimize the co-training dynamics.

2. dt_bias activation: Investigating why dt_bias shows zero gradient signal. Alternative parameterizations may unlock additional capacity.

3. Sparse dimension selection (SDT): Full structured state space dimension tuning — adapting the state transition matrix beyond the per-head scalar constraint.

4. Cross-architecture validation: Replicating this study on other hybrid SSM-Transformer models to establish whether the co-training effect generalizes.

5. Multi-seed validation: Running V3 and V4 training with multiple random seeds to establish confidence intervals on the V3→V4 delta.

8. Conclusion

We demonstrate that effective fine-tuning of hybrid Mamba-Transformer models requires architecture-aware LoRA target selection. Default Transformer module names are catastrophically incorrect for these models, yielding zero improvement despite successful training runs. Correct targeting of Mamba-2 projections and shared MLP layers produces dramatic task improvements (0% to 97%+ accuracy on structured output tasks).

Beyond correct target selection, co-training LoRA adapters with unfrozen SSM core parameters provides two distinct benefits:

1. Co-training effect (V3): Unfreezing SSM parameters during training reshapes the loss landscape, enabling LoRA adapters to converge to better solutions — even when the SSM changes are discarded. This accounts for 76% of the total classification improvement over LoRA-only (V2), and is the primary driver of gains on schema mapping and rule generation tasks.

2. Persistent SSM adaptation (V4): Preserving trained SSM parameter values in the deployed model provides an additional classification boost (+3.6 points), bringing the cumulative improvement to 55.8% exact match (vs 40.8% for LoRA-only) — a 37% relative improvement. This benefit is task-dependent: classification (sequence-level discrimination) benefits from modified state dynamics, while structured output tasks (schema mapping, code generation) are neutral to SSM persistence.

The total cost of SSM co-training is 6,912 additional trainable parameters (0.024% of the LoRA budget) with zero inference overhead, since the SSM parameters already exist in every Mamba-2 layer.

These results establish practical guidelines for fine-tuning the emerging class of hybrid SSM-Transformer models: (1) always inspect `named_modules()` to identify the correct LoRA targets, (2) unfreezing SSM core parameters during training is a free improvement even if persistence is not implemented, and (3) persisting SSM changes provides additional task-dependent gains that are most pronounced for discriminative sequence-level tasks.

Appendix A: Detailed Evaluation Results

Raw evaluation results from 1,000 test examples per task are provided in `eval_results.json` and `eval_results_v4_lora_plus_ssm_fixed.json`.

A.1 Full Results Table

Task	Metric	Base	V1	V2	V3	V4
Classification	exact_match	0.0%	0.1%	40.8%	52.2%	**55.8%**
Classification	contains_match	0.1%	0.1%	45.9%	53.8%	**58.4%**
Mapping	json_valid	1.9%	0.4%	99.9%	**99.9%**	99.5%
Mapping	mapping_accuracy	0.48%	0.0%	96.03%	**97.65%**	96.73%
Mapping	unmapped_accuracy	1.2%	0.3%	96.64%	**97.55%**	97.28%
Rule Generation	json_valid	29.99%	67.94%	98.58%	**98.91%**	98.8%
Rule Generation	all_fields	0.0%	0.0%	98.15%	**98.80%**	97.49%
Rule Generation	severity_correct	3.27%	0.11%	39.91%	**41.33%**	40.57%
Rule Generation	category_label	0.0%	0.0%	2.73%	**3.16%**	3.05%

Note: Bold indicates best result per metric. V3 leads on mapping/code_generation metrics; V4 leads on classification. V3 and V4 mapping/code_generation differences are within evaluation noise.

A.2 Trainable Parameter Comparison

Version	LoRA Params	SSM Core Params	Total Trainable	SSM Persisted	% of Model
V1	851,968	0	851,968	—	0.027%
V2	28,823,552	0	28,823,552	—	0.895%
V3	28,823,552	6,912	28,830,464	No (lost)	0.895%
V4	28,823,552	6,912	28,830,464	**Yes**	0.895%

A.3 SSM Parameter Drift During Training

V1 (Standard LoRA — SSM Frozen)

V1 training logged SSM parameter snapshots before and after training as a control. All parameters showed zero change across all three tasks:

Task	A_log Changed	D Changed	dt_bias Changed
Classification	0.0%	0.0%	0.0%
Schema Mapping	0.0%	0.0%	0.0%
Rule Generation	0.0%	0.0%	0.0%

V2 training used an earlier version of the training script that did not include SSM delta monitoring.

V3/V4 (SSM Unfrozen — Cross-Task Comparison)

SSM parameter drift from training logs (layers 0–2 shown; pattern is consistent across all 36 Mamba layers). V3 and V4 values are near-identical since both use the same training configuration — only the save order differs.

Classification:

Layer	A_log % Changed	D % Changed	dt_bias % Changed
0	18.8%	46.9%	0.0%
1	35.9%	62.5%	0.0%
2	45.3%	93.8%	0.0%

Schema Mapping:

Layer	A_log % Changed	D % Changed	dt_bias % Changed
0	25.0%	39.1%	0.0%
1	40.6%	59.4%	0.0%
2	39.1%	92.2%	0.0%

Structured Rule Generation:

Layer	A_log % Changed	D % Changed	dt_bias % Changed
0	26.6%	68.8%	0.0%
1	45.3%	65.6%	3.1%
2	48.4%	93.8%	0.0%

Cross-Task Summary (Layer 2, representative)

Parameter	Classification	Schema Mapping	Rule Generation	Role
A_log	45.3%	39.1%	48.4%	State transition decay
D	93.8%	92.2%	93.8%	Direct feedthrough
dt_bias	0.0%	0.0%	0.0%	Timestep discretization

V4 Persistence Verification

Task	A_log Mean Abs Diff	SSM Keys Saved
Classification	0.000345	108
Schema Mapping	0.000412	108
Rule Generation	0.000418	108

Key Observations

1. D dominates adaptation: The direct feedthrough parameter consistently shows 2–3x higher change rates than A_log across all tasks and layers (e.g., 93.8% vs 45.3% at layer 2 for classification). The model preferentially tunes how much raw input bypasses the recurrent state, rather than how state transitions decay.

2. dt_bias is effectively frozen: 0.0% change in all but one cell (rule generation, layer 1, 3.1%). The exponential activation applied to dt_bias during the Mamba-2 forward pass likely suppresses its gradient

signal below the optimizer's effective threshold.

3. Deeper layers adapt more: Layer 2 consistently shows higher change rates than layer 0 (e.g., A_log: 45.3% vs 18.8% for classification), suggesting task-specific SSM adaptation concentrates in later layers closer to the output head.

4. Code generation drives the strongest D response: D at layer 0 changes 68.8% for rule generation vs 46.9% for classification and 39.1% for mapping, indicating structured output tasks demand more aggressive state-space reweighting in early layers despite showing no benefit from SSM persistence at inference time.

Appendix B: Software and Hardware

Component	Specification
Training hardware	NVIDIA RTX PRO 6000 96GB
Inference hardware	NVIDIA GeForce RTX 4060 Ti 16GB
Framework	HuggingFace Transformers + PEFT 0.18.1 + SFTTrainer
SSM kernels	mamba_ssm (Triton) + causal-conv1d
Inference runtime	llama-cpp-python 0.3.16 (CUDA 13.1 backend)
GGUF conversion	llama.cpp convert_hf_to_gguf.py
Data generation	LLM-based synthetic pipeline via API
Quantization	Q4_K_M (base), Q8_0 (merged), F16 (LoRA adapters)